

AI-Native Engineer Capstone — Project Brief

ClaudeForge Banking Suite

Business Case BC-AINE-001 · Domain: Banking (Retail / Digital)

1. Project Identity

Business Case Title	ClaudeForge Banking Suite
Business Case ID	BC-AINE-001
Domain	Banking — Retail / Digital
Project Type	AI-Native Engineering Capstone
Cohort	(filled by Operations team)

2. Engagement Overview

Duration	20 hours
Format	Individual
Evaluation Mode	Automated review of submitted Git repository against the AI-Native Engineering Rubric (21 parameters / 100 marks across 5 categories)
Submission	Push final code to your assigned Virtusa GitLab repository by the cohort cut-off
Review Output	Per-learner Excel report (Summary, Categories, Scorecard, Detailed, Improvement)
Grade Bands	Distinction ≥ 90 · Merit 75–89 · Pass 60–74 · Not Yet Passed < 60

What is evaluated: AI-native engineering practices (specs, agents, skills, hooks, CLAUDE.md hierarchy, plugin packaging, MCP, Harness plugin usage), business understanding (context, rules, AC), architectural discipline (structural rules, TDD), technical implementation (code, tests, coverage, UI validation), and CI/CD integration (PR-driven workflow, Claude in CI).

What is not evaluated: the specific technology stack chosen, the visual polish of the UI, or the use of any AI provider beyond Claude Code.

3. Problem Statement & Expected Solution

3.1 Problem

Horizon Bank, a digital-first retail bank, must modernize its customer-facing platform under one constraint: no manual coding. Every line of production code must be generated by Claude Code agents under your supervision.

3.2 Your Role

AI-Native Engineer — architect, supervisor, and quality gatekeeper. You write specifications, configure agents, install guardrails, and review pull requests. You do not write production code by hand.

3.3 Expected Solution

A working banking platform delivered as a Git repository that demonstrates:

- A complete AI-native engineering substrate built on the Claude Harness Engine plugin foundation, extended with banking-specific specs, agents, skills, hooks, commands, and MCP integration.
- A functional banking application built end-to-end by Claude Code agents, with no hand-written production code.
- An evidence trail (PRs, agent logs, post-mortems, sprint contracts) proving the work was agent-led.

3.4 Applicable Rules

- **No-Hand-Coding Rule.** All production code, tests, and migrations must be generated by Claude Code agents. Hand-edits are limited to specs, CLAUDE.md, agent definitions, hooks, skills, and other substrate files.
- **Spec-Is-Truth Rule.** Whenever a spec and code disagree, the spec wins. Update the spec deliberately and regenerate the code.
- **PR-Only Merge Rule.** No direct commits to main. All changes flow through agent-reviewed pull requests.
- **Synthetic-Data Rule.** Use only dummy data. Do not use Virtusa confidential or customer-sensitive data for this project.

4. Technology Stack

Choose any stack that matches your background. The stack does not affect evaluation.

Layer	Options
Frontend	React · Angular · Vue · server-rendered (Thymeleaf / Razor)
Backend	Java + Spring Boot · .NET + ASP.NET Core Web API · Python + FastAPI · Node.js + Express / NestJS
Database	In-memory (H2 / SQLite) · RDBMS (MySQL / MariaDB / SQL Server) · NoSQL (MongoDB / DynamoDB)
Build & Dependency Management	Maven / Gradle · dotnet CLI · pip + Poetry · npm / pnpm
Testing	JUnit / NUnit / pytest / Vitest · Playwright (E2E) · ArchUnit / dependency-cruiser / import-linter
CI	GitLab CI (preferred for Virtusa) · GitHub Actions, with Claude Code Action integrated
MCP Server	Playwright MCP

Layer	Options
Required Plugin	Claude Harness Engine plugin (Generator–Evaluator harness for autonomous long-running development)

5. Acceptance Criteria & Non-Functional Requirements

5.1 Functional Acceptance Criteria

ID	Criterion
AC-01	Customer can register, complete basic KYC stub, and log in
AC-02	Customer can open a savings account; balance is initialized to 0.00
AC-03	Customer can view current balance and statements
AC-04	Customer can transfer money to another account; debit and credit are atomic
AC-05	Transfer fails with <code>InsufficientFundsException</code> when balance is less than amount
AC-06	Customer can add and remove beneficiaries
AC-07	Customer can apply for a personal loan; status flows: <code>APPLIED</code> → <code>UNDER_REVIEW</code> → <code>APPROVED</code> / <code>REJECTED</code> → <code>DISBURSED</code>
AC-08	Admin can list pending loan applications and approve or reject with a reason
AC-09	Admin can view audit log of all transactions and loan state transitions
AC-10	Notification (email or SMS, stub allowed) is sent on successful transfer and on every loan state transition

Note. Each AC must have at least one test referencing its AC-NN identifier.

5.2 Non-Functional Requirements

ID	Requirement
NFR-01	Money is computed in fixed-point (<code>BigDecimal</code> / decimal) — never floating-point
NFR-02	Audit log entries are append-only — no update or delete operations
NFR-03	Card numbers, passwords, and tokens are never written to logs
NFR-04	Authentication boundary enforced at controller layer; no anonymous access to customer / admin APIs
NFR-05	Database migrations or schema changes are append-only
NFR-06	Structured JSON logs with request correlation IDs
NFR-07	Health endpoint returns 200 within 1 second of a successful startup

ID	Requirement
NFR-08	Architecture rules enforced as automated tests (e.g., controllers do not call repositories directly)

6. Functional Scope

6.1 Customer Functionality (User Journeys)

- **Onboarding.** Register → KYC stub → activate login.
- **Account Management.** Open savings account → view balance → list statements.
- **Transfers.** Select beneficiary → enter amount → confirm → see updated balance.
- **Beneficiary Management.** Add / list / remove beneficiaries.
- **Loans.** Apply for personal loan → track status → receive notification on approval / disbursement.

6.2 Internal & System Functionality

- **Admin Dashboard.** Loan application queue (review / approve / reject with reason) and audit log viewer.
- **Notification Module.** Email or SMS dispatch (stub gateway acceptable) on transfer success and loan state transitions.
- **Audit Module.** Append-only ledger of every transaction and state change, with timestamp, actor, and entity reference.

6.3 Out of Scope

You do not need to build:

- Real payment rails / interbank settlement
- Real KYC document verification (stub the response)
- Real SMS or email gateway (log-to-file is acceptable)
- Production deployment, secret management, multi-region, distributed locking
- Penetration testing, threat modelling, regulatory filings

7. AI-Native Implementation Expectations

Foundation. Install the **Claude Harness Engine plugin** first. The harness ships 7 base agents (planner, generator, evaluator, security-reviewer, test-engineer, design-critic, ui-designer), 12+ hooks, base commands, sprint contracts, an evaluator wired to Playwright MCP, and a self-healing loop. On top of that foundation, your repository must add the banking-specific substrate listed in this section. The rubric scores the final repository — both Harness-supplied and learner-added artifacts contribute, but the thresholds below are calibrated so that learners who only install Harness without customizing for the banking domain will not meet them.

7.1 Claude Code Specs (40 marks)

- specs/app_spec.md (root spec) and specs/<feature>_spec.md per major banking feature, each with explicit Acceptance Criteria sections
- Layered CLAUDE.md — root + per-module + per-folder; AGENTS.md as a table of contents only

- At least 2 banking-specific skills under `.claude/skills/` added by you (examples: `banking-rule-generator`, `audit-event-logger`, `money-precision-formatter`)
- At least 2 banking-specific custom commands under `.claude/commands/` added by you (beyond Harness defaults)
- At least 1 banking-specific hook under `.claude/hooks/` added by you (examples: `money-precision-check.sh`, `audit-immutability-check.sh`, `pii-scrub.sh`)
- At least 2 banking-specific agents under `.claude/agents/` added by you (examples: `loan-approval-agent`, `transaction-auditor-agent`). You may rename Harness's generator → `implementer`, evaluator → `reviewer`, test-engineer → `tester` to align with canonical naming
- At least one programmatic Claude Agent SDK usage in scripts/
- Your own `plugin.json` packaging your banking substrate (in addition to Harness's `plugin.json`)
- At least one project-specific debugging-log or post-mortem document under `docs/` evidencing environment-first resolution
- Playwright MCP configured in `.mcp.json` and actively used by the evaluator

7.2 Business Understanding (20 marks)

- `docs/business-case.md` covering: problem, target users, success metrics, domain rules, value proposition. This is the document the reviewer plugin reads — see Section 8
- At least 4 rule / policy / validator files under `src/domain/` (or equivalent) implementing banking business rules
- Frontend evidence (any of: React / Angular / Vue) with at least one responsive layout
- Acceptance criteria expressed in testable form — Given-When-Then or AC-NN identifiers — for at least 4 specs

7.3 Architectural Discipline (8 marks)

- Architecture-test directory (`tests/architecture/` or equivalent) with at least 3 structural tests
- ArchUnit / dependency-cruiser / import-linter configured to enforce layering rules
- TDD discipline document at `docs/tdd.md` and at least 10 test files committed across history showing red–green–refactor pattern

7.4 Technical Implementation (26 marks)

- `docs/architecture.md` with at least 1 architecture diagram (Mermaid or image) and an explicit layered structure (`controllers` / `services` / `repositories` / `domain`)
- Substantive codebase with at least 3000 lines of meaningful generated code
- Playwright (or equivalent) for UI validation with at least one snapshot directory
- Tests tagged with AC IDs — every spec AC has at least 1 corresponding test
- At least 20 unit tests with a coverage artefact (`coverage.xml`, `lcov.info`, or equivalent) and the coverage tool declared in dependencies

7.5 CI/CD (6 marks)

- `.gitlab-ci.yml` (or `.github/workflows/`) with the build and test pipeline
- Claude Code Action (or equivalent) wired into the CI workflow

- At least 3 PR-driven merges (use `git merge --no-ff` to preserve PR history); zero direct commits to main

8. Expected Outcomes & Deliverables

By the end of 20 hours, the submitted Git repository must contain the Mandatory items below.

Good-to-Have items differentiate Merit and Distinction submissions.

8.1 Mandatory

1. **Working Application.** Banking platform deployable locally via a single command (`./mvnw spring-boot:run`, `npm start`, `dotnet run`, or equivalent) with seed data and a README quick-start.
2. **docs/business-case.md.** The business context document the reviewer plugin reads. Use this brief's Sections 3 and 6 as the source; rewrite in your own words covering problem, users, success metrics, domain rules, and value proposition.
3. **Specifications.** `specs/app_spec.md` plus per-feature specs with explicit Acceptance Criteria sections.
4. **Layered CLAUDE.md Hierarchy.** Root + per-module + per-folder; `AGENTS.md` as a table of contents.
5. **Banking Agent Substrate.** `.claude/agents/`, `.claude/skills/`, `.claude/commands/`, `.claude/hooks/` — Harness foundation plus your banking-specific additions as listed in Section 7.1.
6. **Claude Harness Engine Plugin Integration.** Plugin installed; project scaffolded via the harness; at least one full sprint cycle (Generator → Evaluator → ratcheting → PR) driven through it. Sprint contracts and evaluator outputs committed under `sprint-contracts/` and `specs/reviews/`.
7. **Plugin Manifest.** Your own `plugin.json` at the repository root packaging your banking substrate; `.mcp.json` with Playwright MCP active.
8. **Test Suite.** Unit tests, architecture tests, AC-tagged tests, Playwright UI tests, coverage report — all committed.
9. **CI Pipeline.** `.gitlab-ci.yml` (or equivalent) with Claude Code Action wired in; passing build on main.
10. **PR-Driven Git History.** At least 3 PR-driven merges; zero direct pushes to main.

8.2 Good-to-Have

11. **Architecture & TDD Documentation.** `docs/architecture.md` with at least one diagram; `docs/tdd.md` describing the red–green–refactor approach.
12. **Autonomous Fix Loop Evidence.** At least one recorded loop where an agent detected a failure, reproduced it, fixed, validated, and opened a PR. Capture the trace under `docs/fix-loops/`.
13. **Knowledge Deposit Log.** `docs/knowledge-deposits.md` recording recurring mistakes encoded back into rules, hooks, or skills.
14. **Multiple Sprint Cycles.** More than one harness-driven sprint, demonstrating ratcheting across iterations.
15. **Bonus Agents.** Beyond the required additions: janitor, performance-auditor, doc-writer, or other domain-specific agents.

On completion of the capstone, you will have demonstrated the skill the industry is seeking: building a real banking application without writing the code yourself, while maintaining full control of quality, security, and architecture through AI-native engineering practices.